

SE 314: Software Construction

Abstract Data Types

Outline

- **What Abstraction means**
- **Classifying Types and Operations**
- **Designing an Abstract Type**
- Representation Independence
- Realizing ADT Concepts in java
- Testing an Abstract Data Type

What Abstractions Means

- Abstract data types are an instance of a general principle in software engineering, which goes by many names with slightly different shades of meaning:
 - Abstraction
 - Modularity
 - Encapsulation
 - Information hiding
 - Separation of concerns

What is an Abstract Data Type in Java?

- We know that a data type signifies the type and space taken by the data used in programs. An Abstract Data Type is a special data type that is defined by a set of values and a set of operations on that type.
- We call these data types as “**abstract**” because these are independent of any implementation. We can use these data types and perform different operations with them, but we do not know how these operations are working internally.
- The implementation details of these data types are totally invisible to the users. It does not specify how the data stores in the memory area and what algorithms are useful for implementing the operations on the data.
- An Abstract Data Type in Java is useful in the implementation of data structures. Java library provides various Abstract Data Types such as List, Stack, Queue, Set, Map as inbuilt interfaces that we implement using various data structures.

User Defined Types

- A type is characterized by the operations you can perform on it.
- The user of the type does not need to worry about how values are actually stored.
- Built-in types vs. User-defined types

Classifying Types and Operations

- Types, whether built-in or user-defined, can be classified as mutable or immutable.
- Date
- String
- StringBuilder

Types and Operations of Java Abstract Data Type

- If an Abstract Data Type is mutable then we can change the objects of its type and if it is immutable then we can't change its object.
- For example, the Date class is mutable because we can call its setMonth() method and observe the change with the getMonth() operation. But String is immutable because its operations do not change the existing objects but create new String objects

Classifying Types and Operations

- The operations of an abstract type are classified as follows:
 - Creators
 - Producers
 - Observers
 - Mutators

Operations

- There are following types of operations of an abstract type:
- **Creators:** Creators create new objects of the type. It may take an object as an argument.
- **Producers:** Producers create new objects from old objects of the type. For example, the `concat()` method of the `String` is a producer that takes two strings and produces a new `String` representing their concatenation.
- **Observers:** Observers take the objects of the abstract type and return objects of a different type. For example, the `size()` method of the `List` returns an `int`.
- **Mutators:** Mutators change objects. For example, the `add()` method of `List` changes a list by adding an element to the end.

Java Abstract Data Type Examples

- Below are some examples of abstract data types, along with some of their operations and the types.
- 1. **int** is a primitive integer type of Java. **int** is **immutable**, so it has no mutators. Its operations are:
 - **creators:** The numeric literals 0, 1, 2, 3,...
 - **producers:** Arithmetic operators +, -, ×, ÷
 - **observers:** Comparison operators ==, !=, <, >
 - **mutators:** None (it's immutable)

Java Abstract Data Type Examples

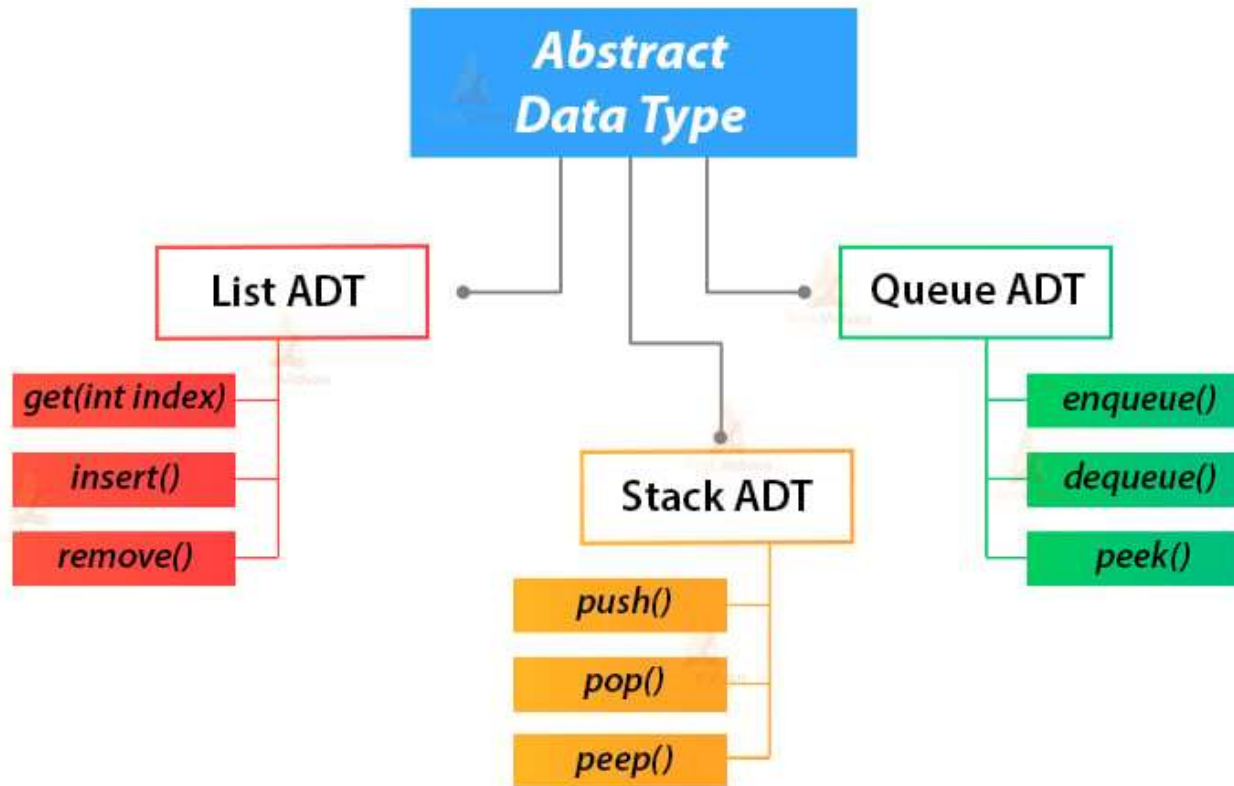
- The **list** is an interface of Java List. The list is **mutable**. Its operations are:
- **creators**: ArrayList and LinkedList constructors, Collections.singletonList
- **producers**: Collections.unmodifiableList
- **observers**: size, get
- **mutators**: add, remove, addAll, Collections.sort

Java Abstract Data Type Examples

- A **string** is Java's string type. The string is **immutable**. Its operations are:
- **creators**: String constructors
- **producers**: concat, substring, toUpperCase
- **observers**: length, charAt
- **mutators**: none (it's immutable)

List of Java Abstract Data Type

Various Java Abstract Data Types



1. List ADT

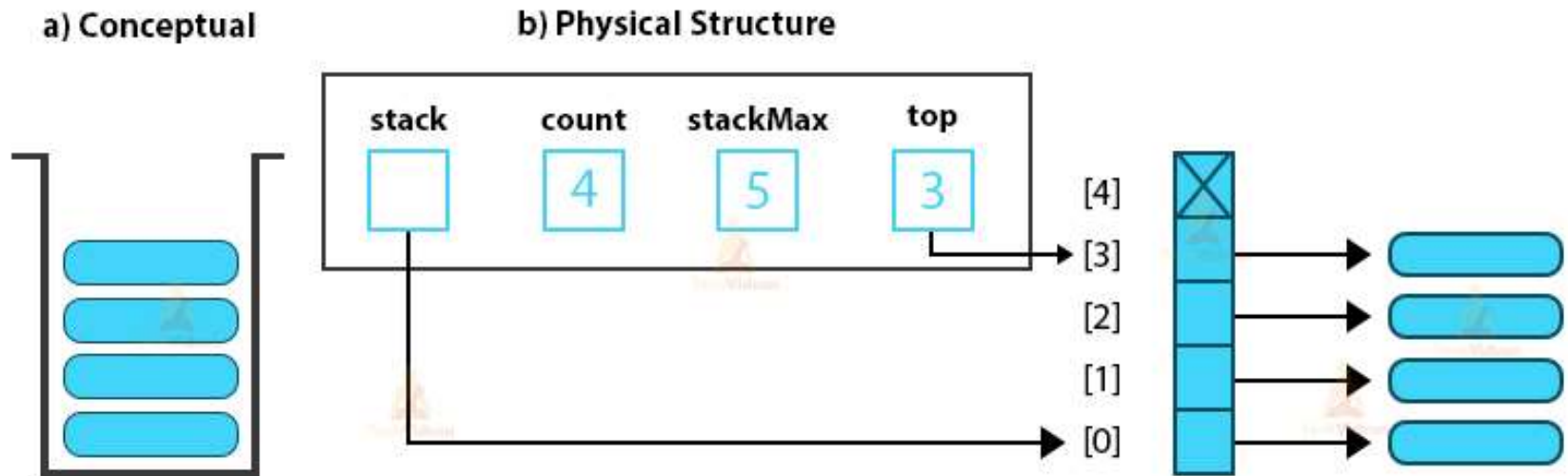
- The List Abstract Data Type is a type of list that contains similar elements in sequential order. The list ADT is a collection of elements that have a linear relationship with each other. A linear relationship means that each element of the list has a unique successor.
- The List ADT is an interface, that is, other classes give the actual implementation of the data type. For example, Array Data Structure internally implements the **ArrayList** class while the List Data Structure internally implements the **LinkedList** class.
- The List [interface of Java](#) library specifies 25 different operations/methods. Following are some of the operations that we can perform on the list:
- **get(int index)**: Returns an element at the specified index from the list.
- **insert()**: Inserts an element at any position.
- **remove()**: Removes the first occurrence of any element from a list.
- **removeAt()**: Removes the element at a predefined area from a non-empty list.
- **Replace()**: Replaces an element by another element.
- **size()**: Returns the number of elements of the list.
- **isEmpty()**: Returns true if the list is empty, else returns false.
- **isFull()**: Returns true if the list is full, else returns false.

2. Stack ADT

- A stack is a LIFO (“Last In, First Out”) data structure that contains similar elements arranged in an ordered sequence. All the operations in stack take place at the **top** of the stack.
- **Stack ADT** is a collection of homogeneous data items (elements), in which all insertions and deletions occur at one end, called the top of the stack.
- In Stack ADT Implementation, there is a pointer to the data, instead of storing the data at each node.
- The program allocates the memory for the data and passes the address to the stack ADT.
- The start node and the data nodes encapsulate together in the ADT. Only the pointer to the stack is visible to the calling function.
- The stack head structure also contains a pointer to the top of the stack and also the count of the number of entries currently in the stack.

2. Stack ADT

Structure of Stack Abstract Data Type in Java



2. Stack ADT

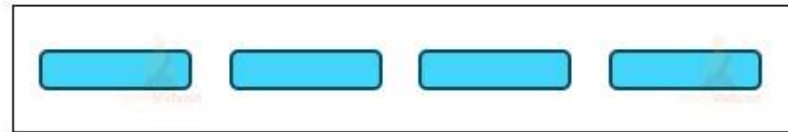
- We can perform the following operations on the stack –
- **push()**: It inserts an element at the top of the stack if the stack is not full.
- **pop()**: It removes or pops an element from the top of the stack if the stack is not empty.
- **peek()**: Returns the top element of the stack without removing it.
- **size()**: Returns the size of the stack.
- **isEmpty()**: If the stack is empty it returns true, else it returns false.
- **isFull()**: If the stack is full it returns true, else it returns false.

3. Queue ADT

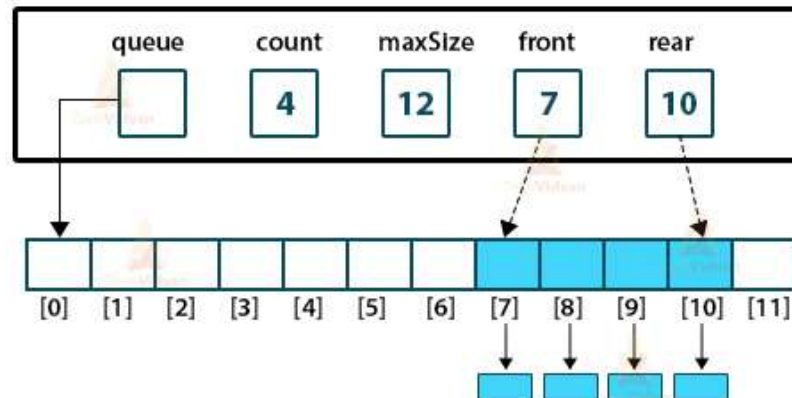
- A Queue is a FIFO (“First In, First Out”) data structure that contains similar types of elements arranged sequentially. We can perform the operations on a queue at both ends; insertion takes place at rear end deletion takes place at the front end.
- **Queue ADT** is a collection in which the arrangement of the elements of the same type is in a sequential manner.
- The design of the Queue abstract data type (ADT) is the same as the basic design of the Stack ADT.
- Each node of the queue contains a void pointer to the data and a link pointer to the next element of the queue. The program allocates the memory for storing the data.

3. Queue ADT

Structure of Queue Abstract Data Type in Java



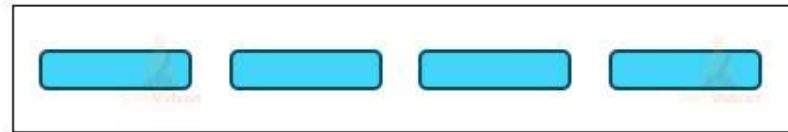
a) Conceptual



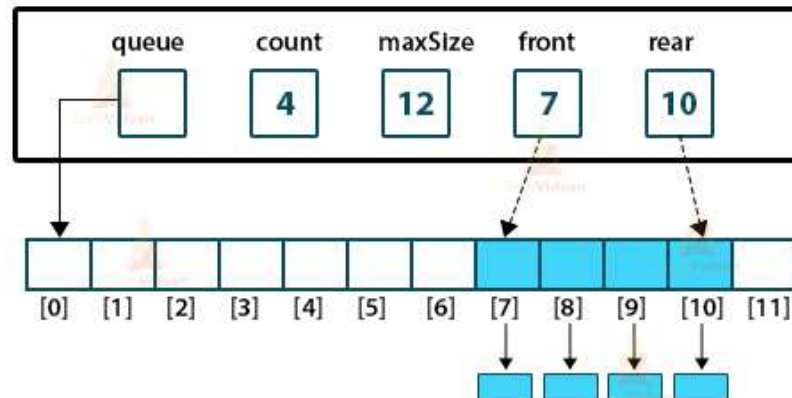
b) Physical Structures

3. Queue ADT

Structure of Queue Abstract Data Type in Java



a) Conceptual



b) Physical Structures

3. Queue ADT

- Operations performed on the queue are as follows:
- **enqueue()**: It inserts or adds an element at the end of the queue.
- **dequeue()**: Removes an element from the front side of the queue.
- **peek()**: Returns the starting element of the queue without removing it.
- **size()**: This function returns the number of elements in the queue.
- **isEmpty()**: If the queue is empty, it returns true, otherwise it returns false.
- **isFull()**: If the queue is full, it returns true, otherwise it returns false.

Designing an Abstract Data Type in Java

- To design an abstract data type we have to choose good operations and determine how they should behave. Here are a few rules for designing an ADT.
- It's better to combine simple and few operations in powerful ways, rather than a lot of complex operations.
- Each operation in an Abstract Data Type should have a clear purpose and should have a logical behavior rather than a range of special cases. All the special cases would make operation difficult to understand and use.
- The set of operations should be adequate so that there are enough kinds of computations that users likely want to do.
- The type may be either generic, for example, a graph, a [list](#) or a set, or it may be domain-specific for example, an employee database, a street map, a phone book, etc. But there should not be a combination of generic and domain-specific features.

Which Java Abstract Data Type to choose?

- Now after having brief knowledge of Java Abstract Data Types, we will discuss the scenarios to choose between either of List, Stack or Queue ADT.
- **List ADT** is a collection of elements and stores them sequentially and which we can access using their indices. We can opt for this ADT in cases that involve indexed or sequential access or removal of elements.
- For example, we can use various implementations of List ADT to store data of a list of employees in sorted order for sequential access or removal.
- A Stack is a Last In First out data structure, and thus we can use implementations of **Stack ADT** in scenarios where we need to firstly access the most recently inserted elements.
- For example, the function call stack of every programming language has the common requirement of this kind of LIFO data structure where there is a need to execute the most recent function in the stack.
- The queue is a First In First Out data structure and we can choose the **Queue ADT** in scenarios where we need to access the elements in their order of insertion.
- For example, one such scenario is request handling by web servers. Web servers make it possible to ensure the fairness of request handling according to their order of arrival by maintaining an internal queue for the requests.

Rep Invariant and Abstraction Function

- We now take a deeper look at the theory underlying abstract data types. If you understand the theory deeply, you'll be able to build better abstract types, and will be less likely to fall into subtle traps.
- In thinking about an abstract type, it helps to consider the relationship between two spaces of values.
- The space of representation values (or rep values for short) consists of the values of the actual implementation entities. In simple cases, an abstract type will be implemented as a single object, but more commonly a small network of objects is needed, so this value is actually often something rather complicated. For now, though, it will suffice to view it simply as a mathematical value.

Rep Invariant and Abstraction Function

- The space of abstract values consists of the values that the type is designed to support. These are a figment of our imaginations. They're platonic entities that don't exist as described, but they are the way we want to view the elements of the abstract type, as clients of the type. For example, an abstract type for unbounded integers might have the mathematical integers as its abstract value space; the fact that it might be implemented as an array of primitive (bounded) integers, say, is not relevant to the user of the type.
- Now of course the implementor of the abstract type must be interested in the representation values, since it is the implementor's job to achieve the illusion of the abstract value space using the rep value space.

Rep Invariant and Abstraction Function

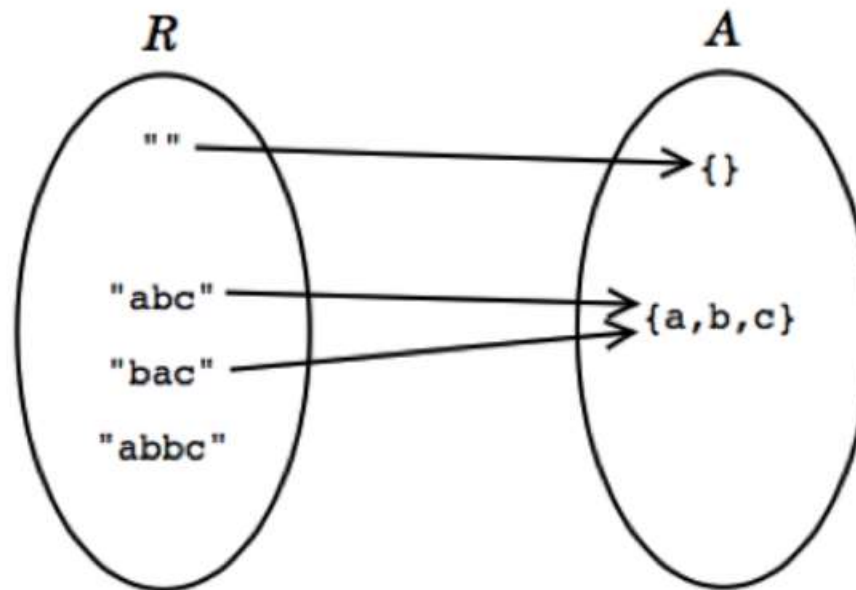
- Suppose, for example, that we choose to use a string to represent a set of characters:

```
public class CharSet  
{ private String s; ... }
```

Then the rep space R contains Strings, and the abstract space A is mathematical sets of characters.

Rep Invariant and Abstraction Function

- We can show the two value spaces graphically, with an arc from a rep value to the abstract value it represents.



Rep Invariant and Abstraction Function

- **Every abstract value** is mapped to by some rep value. The purpose of implementing the abstract type is to support operations on abstract values. Presumably, then, we will need to be able to create and manipulate all possible abstract values, and they must therefore be representable.
- **Some abstract values** are mapped to by more than one rep value. This happens because the representation isn't a tight encoding. There's more than one way to represent an unordered set of characters as a string.
- **Not all rep values** are mapped. In this case, the string "abbc" is not mapped, because we have decided that the rep string should not contain duplicates. This will allow us to terminate the remove method when we hit the first instance of a particular character, since we know there can be at most one.

Rep Invariant and Abstraction Function

- An *abstraction function* that maps rep values to the abstract values they represent:

$$AF : R \rightarrow A$$

The arcs in the diagram show the abstraction function.

- A *rep invariant* that maps rep values to booleans:

$$RI : R \rightarrow \text{boolean}$$

For a rep value r , $RI(r)$ is true if and only if r is mapped by AF .

Rep Invariant and Abstraction Function

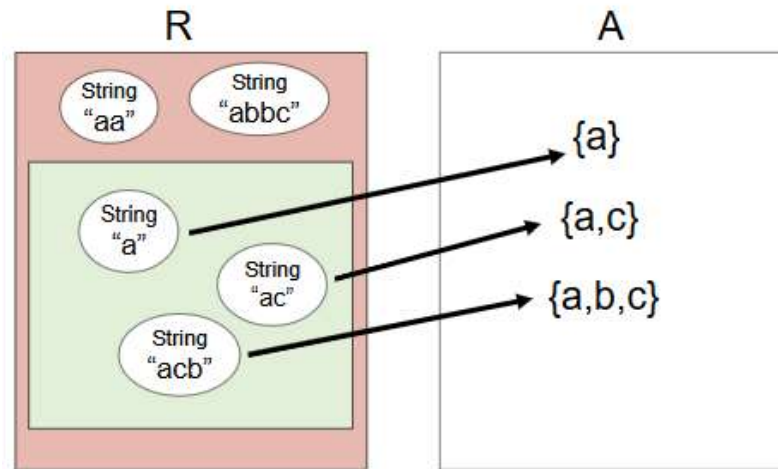
- Both the rep invariant and the abstraction function should be documented in the code.

```
public class CharSet
{
    private String s;
    // Rep invariant:
    // s contains no repeated characters
    // Abstraction Function:
    // represents the set of characters found in s ...
}
```

Rep Invariant and Abstraction Function

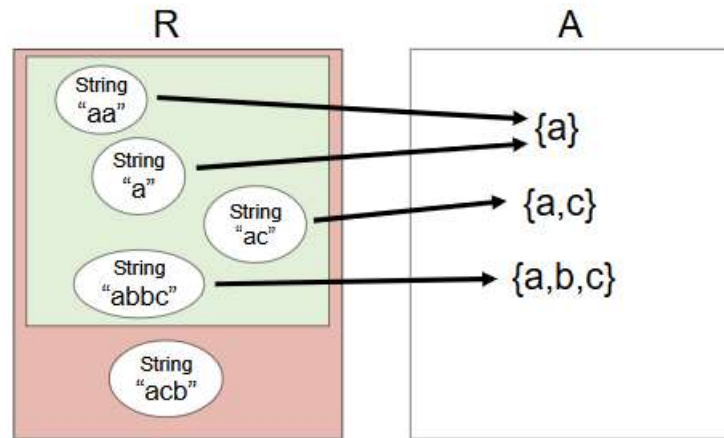
- For a rep value r , $RI(r)$ is true if and only if r is mapped by AF . In other words, RI tells us whether a given rep value is well-formed. Alternatively, you can think of RI as a set: it's the subset of rep values on which AF is defined.
- For example, the diagram at the right showing a rep for CharSet that forbids repeated characters, $RI("a") = \text{true}$, $RI("ac") = \text{true}$, and $RI("acb") = \text{true}$, but $RI("aa") = \text{false}$ and $RI("abbc") = \text{false}$. Rep values that obey the rep invariant are shown in the green part of the R space, and must map to an abstract value in the A space. Rep values that violate the rep invariant are shown in the red zone, and have no equivalent abstract value in the A space.

Rep Invariant and Abstraction Function



```
public class CharSet { private String s;  
  // Rep invariant:  
  // s contains no repeated characters  
  // Abstraction function:  
  //  $AF(s) = \{s[i] \mid 0 \leq i < s.length()\} \dots$  }
```


Rep Invariant and Abstraction Function



```
public class CharSet { private String s;  
  // Rep invariant:  
  // s[0] <= s[1] <= ... <= s[s.length()-1]  
  // Abstraction function: // AF(s) = {s[i] | 0 <= i <  
  s.length()} ... }
```

Stack ADT: Rep Invariant & Abstraction Function

- **public class Stack {**
- private int[] elements;
- private int top;
-
- **// Rep Invariant:**
- // $0 \leq \text{top} \leq \text{elements.length}$
- // elements[0..top-1] contains the stack elements
- // elements[top..elements.length-1] are unused
-
- **// Abstraction Function:**
- // $\text{AF}(\text{elements}, \text{top}) = \text{Stack} \langle \text{elements}[0], \text{elements}[1], \dots, \text{elements}[\text{top}-1] \rangle$
- // where elements[top-1] is the top of the stack
- **}**

Stack ADT: Rep Invariant & Abstraction Function

Rep Invariant:

$0 \leq \text{top} \leq \text{capacity}$, valid elements stored from index 0 to $\text{top}-1$

Abstraction Function:

AF = sequence of elements $[0..\text{top}-1]$, with elements $[\text{top}-1]$ as top

Any Questions